



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: Carandia Lorenzo Brenda Fernanda

N° de Cuenta: 319018961

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 22 de marzo de 2025

CALIFICACIÓN: _____

OPTIMIZACIÓN Y CARGA DE MODELOS

Actividades realizadas

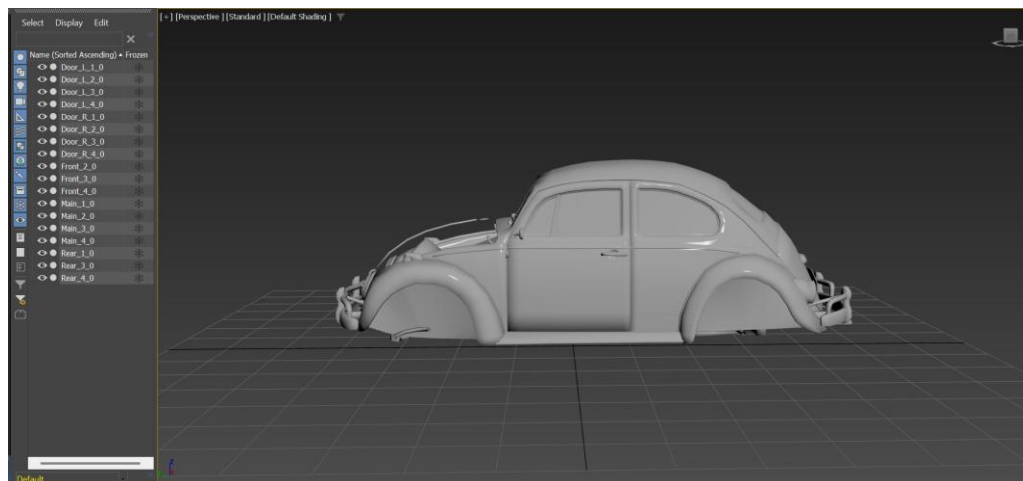
1. Importar su modelo de coche propio dentro del escenario a una escala adecuada.
2. Importar sus 4 llantas y acomodarlas jerárquicamente, agregar el mismo valor de rotación a las llantas para que al presionar puedan rotar hacia adelante y hacia atrás.
3. Importar el cofre del coche, acomodarlo jerárquicamente y agregar la rotación para poder abrir y cerrar.
4. Agregar traslación con teclado para que pueda avanzar y retroceder de forma independiente

Mi coche que elegí fue el siguiente

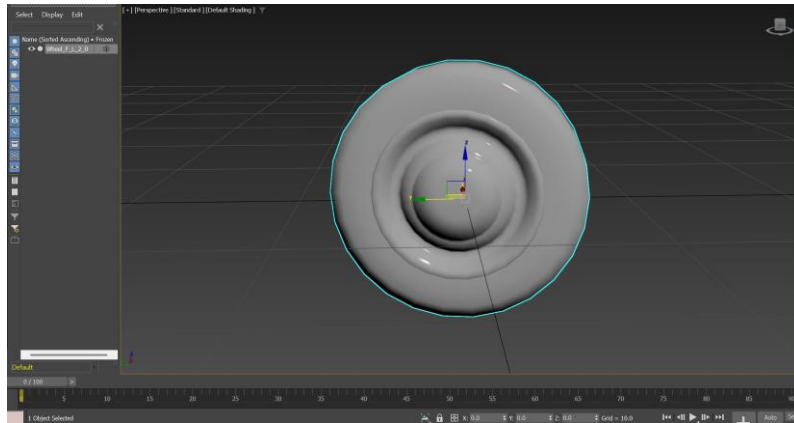


- IMPORTAMOS LAS PARTES DEL COCHE

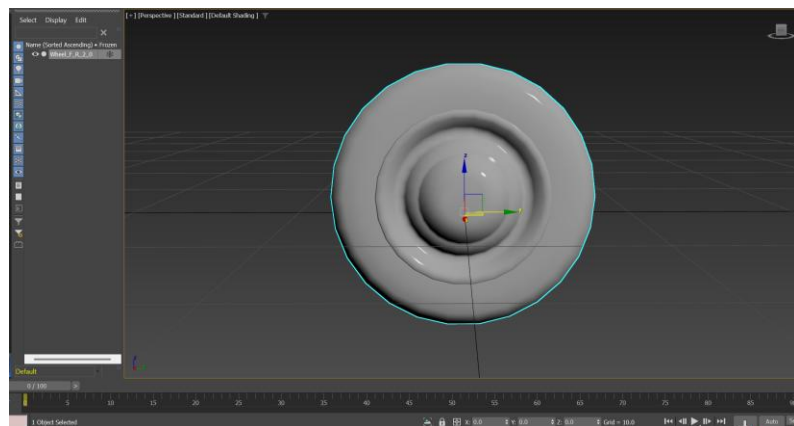
Cuerpo del coche



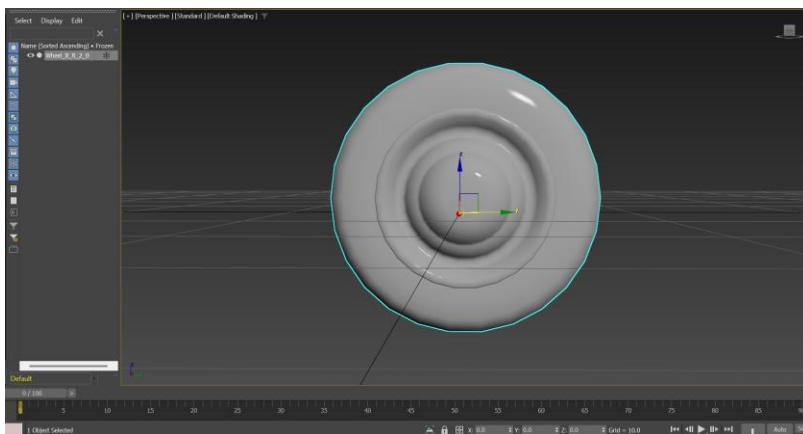
Llanta Delantera Izquierda



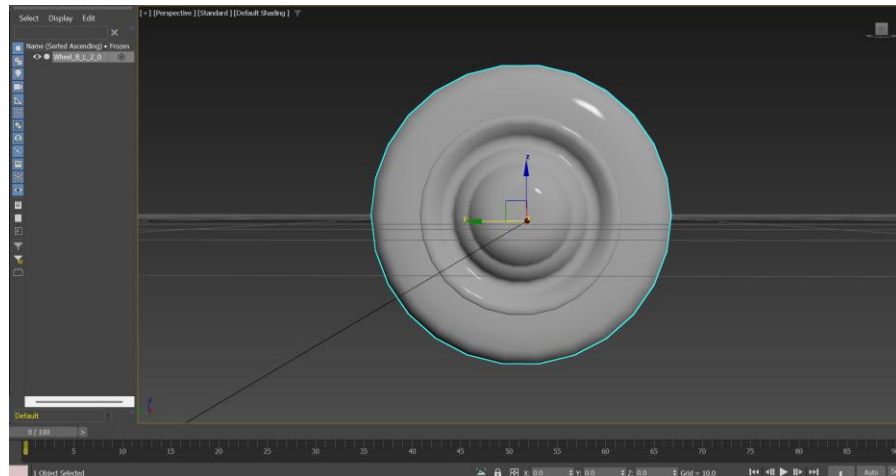
Llanta Delantera Derecha



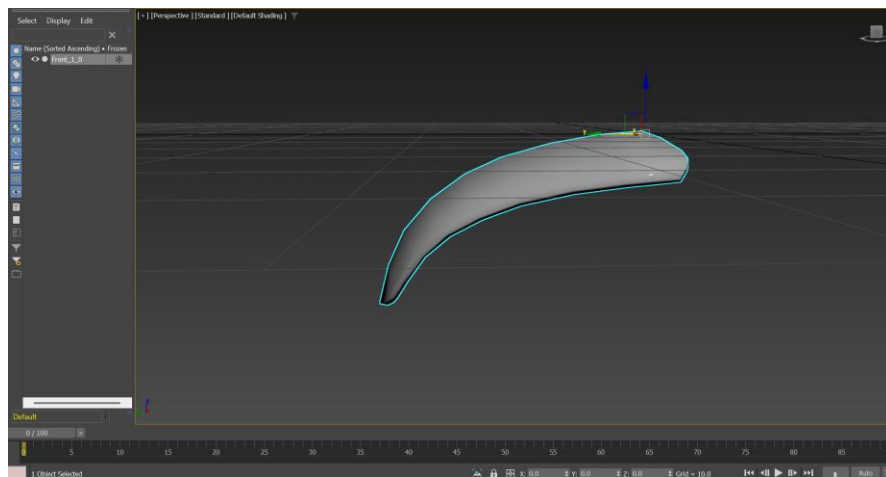
Llanta Trasera Derecha



Llanta Delantera Izquierda



Cofre



En este caso procure que al exportarlos estuvieran en el origen, así mismo ubique a su pivote en la manera en que se tendrían que rotar los elementos. En este caso para las llantas, ya estaban agrupadas todas las partes, por lo que solo las exporte.

CÓDIGO

Window.h

```
#include<glew.h>
#include<glfw3.h>

class Window
{
public:
    Window();
    Window(GLint windowWidth, GLint windowHeight);
    int Initialise();
    GLfloat getBufferWidth() { return bufferWidth; }
    GLfloat getBufferHeight() { return bufferHeight; }
    bool getShouldClose() {
        return glfwWindowShouldClose(mainWindow);
    }
    bool* getsKeys() { return keys; }
    GLfloat getXChange();
    GLfloat getYChange();
    void swapBuffers() { return glfwSwapBuffers(mainWindow); }
    GLfloat getrotay() { return rotay; }
    GLfloat getrotax() { return rotax; }
    GLfloat getrotaz() { return rotaz; }
    // Función para obtener la rotación de las llantas
    GLfloat getrotacionLlantas() { return rotacionLlantas; }

    // Función para obtener la traslación del coche
    GLfloat getmovercarro() { return movercarro; }
    GLfloat getrotacionLlantasIndependiente(int index) {
        if (index >= 0 && index < 4) {
            return rotacionLlantasIndependiente[index];
        }
        return 0.0f;
    }

    // Función para actualizar las rotaciones y traslaciones
    void ActualizarRotaciones(GLfloat deltaTime);

    // Destructor
    ~Window();

private:
    GLFWwindow* mainWindow;
    GLint width, height;

    // Variables para rotaciones generales
    GLfloat rotax, rotay, rotaz, cofre, movercarro;

    // Variables para la rotación de las llantas y traslación del coche
    GLfloat rotacionLlantas; // Rotación común para las llantas
    GLfloat rotacionLlantasIndependiente[4]; // Rotación independiente de las llantas

    bool keys[1024];
    GLint bufferWidth, bufferHeight;
    GLfloat lastX;
    GLfloat lastY;
    GLfloat xChange;
    GLfloat yChange;
    bool mouseFirstMoved;
    void createCallbacks();
    static void ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode);
    static void ManejaMouse(GLFWwindow* window, double xPos, double yPos);
};
```

Se agregan las variables de rotación para cada llanta tanto para que se mueva de manera dependiente como de manera independiente, así como para trasladar el carro; y la función de `ActualizarRotaciones` para limitarlo.

Window.cpp

```
Window::Window(GLint windowHeight, GLint windowHeight)
{
    width = windowHeight;
    height = windowHeight;
    rotax = 0.0f;
    rotay = 0.0f;
    rotaz = 0.0f;
    rotacionLlantas = 0.0f; // Rotación para las llantas
    movercarro = 0.0f; // Traslación del coche
    cofre = true;

    for (int i = 0; i < 4; i++) {
        rotacionLlantasIndependiente[i] = 0.0f; // Inicializar rotación independiente de cada llanta
    }

    for (size_t i = 0; i < 1024; i++) {
        keys[i] = 0;
    }
}
```

Inicializamos las variables que vamos a utilizar

```
WINDOW* theWindow = static_cast<WINDOW*>(&glfwGetWindowSetPointer[theWindow]);

if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS)
{
    glfwSetWindowShouldClose(window, GL_TRUE);
}

if (key == GLFW_KEY_R)
{
    theWindow->rotay += 10.0; // Rotar sobre el eje y 10 grados
}
if (key == GLFW_KEY_T)
{
    theWindow->rotaz += 10.0;
}
//Traslacion coche
if (key == GLFW_KEY_B && (action == GLFW_PRESS || action == GLFW_REPEAT)) {
    theWindow->movercarro += 5.0f; // Mover hacia atras
    theWindow->rotacionLlantas += 10.0f;
}
if (key == GLFW_KEY_N && (action == GLFW_PRESS || action == GLFW_REPEAT)) {
    theWindow->movercarro -= 5.0f; // Mover hacia adelante
    theWindow->rotacionLlantas -= 10.0f;
}
// Rotación independiente de las llantas (adelante)
if (key == GLFW_KEY_G && (action == GLFW_PRESS || action == GLFW_REPEAT)) {
    for (int i = 0; i < 4; i++) {
        theWindow->rotacionLlantasIndependiente[i] += 5.0f; // Girar todas las llantas a la derecha
    }
}

// Rotación independiente de las llantas (atras)
if (key == GLFW_KEY_H && (action == GLFW_PRESS || action == GLFW_REPEAT)) {
    for (int i = 0; i < 4; i++) {
        theWindow->rotacionLlantasIndependiente[i] -= 5.0f; // Girar todas las llantas a la izquierda
    }
}
```

En esta parte gestionamos las entradas de teclado, con las teclas B y N controlamos el movimiento hacia atrás y adelante del coche, es decir, la translación, ajustando también la rotación de las llantas. Además, las teclas G y H permiten girar todas las llantas de manera independiente, simulando un giro hacia adelante o hacia atrás.

```

void Window::ActualizarRotaciones(GLfloat deltaTime)
{
    GLfloat velocidadRotacion = 5.0f;

    // rotación de llantas con movimiento en el carro
    if (rotacionLlantas > 360.0f) {
        rotacionLlantas -= 360.0f;
    }
    else if (rotacionLlantas < -360.0f) {
        rotacionLlantas += 360.0f;
    }

    // rotación independiente para cada llanta
    for (int i = 0; i < 4; i++) {
        if (rotacionLlantasIndependiente[i] > 360.0f) {
            rotacionLlantasIndependiente[i] = 360.0f; // Límite máximo de giro de las llantas
        }
        else if (rotacionLlantasIndependiente[i] < -360.0f) {
            rotacionLlantasIndependiente[i] = -360.0f; // Límite máximo de giro de las llantas
        }
    }

    //Cofre
    if (keys[GLFW_KEY_E])
    {
        if (cofre)
        {
            rotax += velocidadRotacion * deltaTime; // Rotar hacia adelante
            if (rotax > 45.0f)
            {
                rotax = 45.0f; // Limitar a 45°
                cofre = false; // Cambiar dirección
            }
        }
        else
        {
            rotax -= velocidadRotacion * deltaTime; // Rotar hacia atrás
            if (rotax < 0.0f)
            {
                rotax = 0.0f; // Limitar a 0°
                cofre = true; // Cambiar dirección
            }
        }
    }
}

```

Esta función se encarga de actualizar las rotaciones del coche y del cofre. La rotación de las llantas está limitada entre -360° y 360. Además, la rotación independiente de cada llanta también se mantiene dentro de estos límites, asegurando que no exceda los rangos establecidos. Por otro lado, tenemos la rotación del cofre, que se abre y cierra gradualmente al presionar la tecla E. La rotación del cofre está limitada entre 0° y 45°, y cambia de dirección automáticamente cuando alcanza estos valores. Para lograr un movimiento natural, se utiliza una velocidad de rotación fija (5.0f), que se multiplica por deltaTime.

Main

```

CarroCuerpo_M = Model();
CarroCuerpo_M.LoadModel("Model carro/cuerpocarro.obj");
llantaDIzq_M = Model();
llantaDIzq_M.LoadModel("Model carro/llantaizqDelantera.obj");
llantaDDer_M = Model();
llantaDDer_M.LoadModel("Model carro/llantaderDelantera.obj");
llantaTDer_M = Model();
llantaTDer_M.LoadModel("Model carro/llantaderTrasera.obj");
llantaTIzq_M = Model();
llantaTIzq_M.LoadModel("Model carro/llantaizqTrasera.obj");
cofre_M = Model();
cofre_M.LoadModel("Model carro/cofre.obj");

```

Cargamos nuestros modelos de cada parte de nuestro auto, que están guardados en nuestra carpeta Model carro. Así como inicializamos nuestra matriz model y modelaux que nos ayudara con nuestra jerarquía.

```
//COCHE

// 1. Dibujar el cuerpo del coche con traslación
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 2.0f, mainWindow.getmovercarro())); //Traslación del coche en Z
modelaux = model;
color = glm::vec3(0.0f, 0.502f, 0.502f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
CarroCuerpo_M.RenderModel();

// 2. Dibujar cofre (hija del cuerpo)
model = modelaux; // Heredar la transformación del cuerpo
model = glm::translate(model, glm::vec3(1.1f, 22.06f, -13.0f)); // Acomodarlo al cuerpo
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotación del cofre E
color = glm::vec3(0.0f, 0.0f, 0.0f); // Color del cofre
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cofre_M.RenderModel();

// 4. Dibujar las LLANTAS (hijas del cuerpo)
// Llanta delantera derecha
model = modelaux; // Restaurar la matriz base del cuerpo
model = glm::translate(model, glm::vec3(15.0f, 2.6f, -24.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotacionLlantas() + mainWindow.getrotacionLlantasIndependiente(0)), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotación de la llanta
color = glm::vec3(1.0f, 0.980f, 0.941f); // Color de la llanta
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantaDDer_M.RenderModel();

// Llanta delantera izquierda
model = modelaux; // Restaurar la matriz base del cuerpo
model = glm::translate(model, glm::vec3(-13.5f, 3.2f, -24.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotacionLlantas() + mainWindow.getrotacionLlantasIndependiente(1)), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotación de la llanta
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantaDIZq_M.RenderModel();

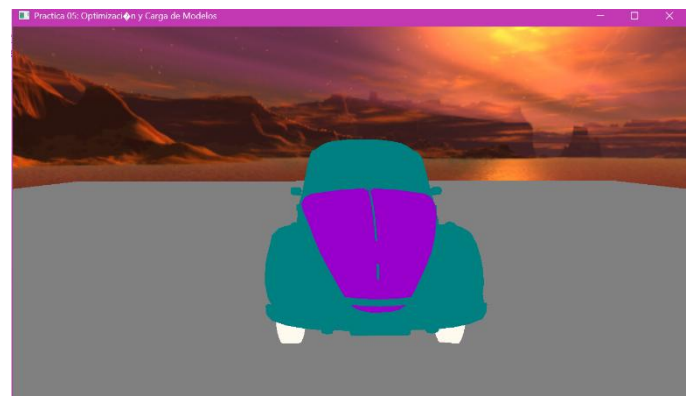
// Llanta trasera derecha
model = modelaux; // Restaurar la matriz base del cuerpo
model = glm::translate(model, glm::vec3(16.0f, 3.1f, 28.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotacionLlantas() + mainWindow.getrotacionLlantasIndependiente(2)), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotación de la llanta
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantaTDer_M.RenderModel();

// Llanta trasera izquierda
model = modelaux; // Restaurar la matriz base del cuerpo
model = glm::translate(model, glm::vec3(-14.2f, 3.4f, 29.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotacionLlantas() + mainWindow.getrotacionLlantasIndependiente(3)), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotación de la llanta
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantaTIZq_M.RenderModel();

glUseProgram(0);
```

En esta parte hacemos las transformaciones necesarias para volver a unir las llantas y el cofre de manera correcta, de acuerdo con el cuerpo del auto, que es nuestro padre. Utilizamos la matriz modelaux para ayudarnos de la jerarquía, todas las llantas son hijos del cuerpo. Y utilizamos rotación y traslación para simular el movimiento del auto.

- Ejecución del programa.



- **Problemas presentados.**

En esta práctica no se me presentaron problemas a la hora de ejecutar el código. Solo un momento a la hora de la rotación de una llanta, pero se pudo corregir al poner en la posición correcta el pivote.

- **Conclusiones**

Estos ejercicios fueron de gran ayuda para reforzar el tema de la jerarquía de los elementos y poder comprenderlos mejor. También me ayudo para familiarizarme un poco mas con las funciones de 3D Max para poder escalar los elementos, agruparlos o desagruparlos para poder exportarlos. También nos ayudo a ver como implementar las rotaciones a diferentes elementos con un cierto limite, y lo nuevo para mí, fue el poder simular el avance del coche de la traslación. Me parecieron con la complejidad adecuada.

- **Bibliografía**

✚ Códigos de colores HTML. (s. f.). HTML Color Codes. <https://htmlcolorcodes.com/es/>

✚ LearnOpenGL - Model. (s. f.). <https://learnopengl.com/Model-Loading/Model>